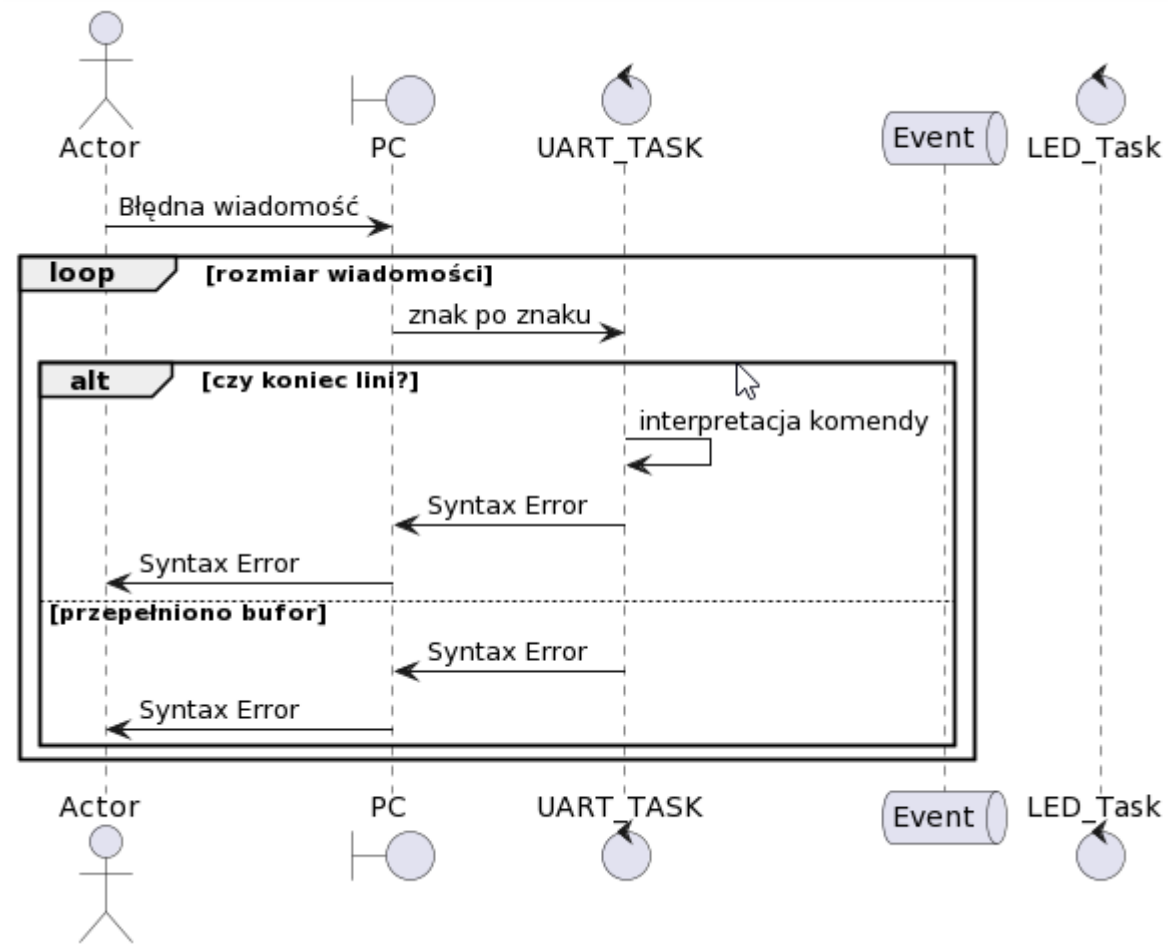
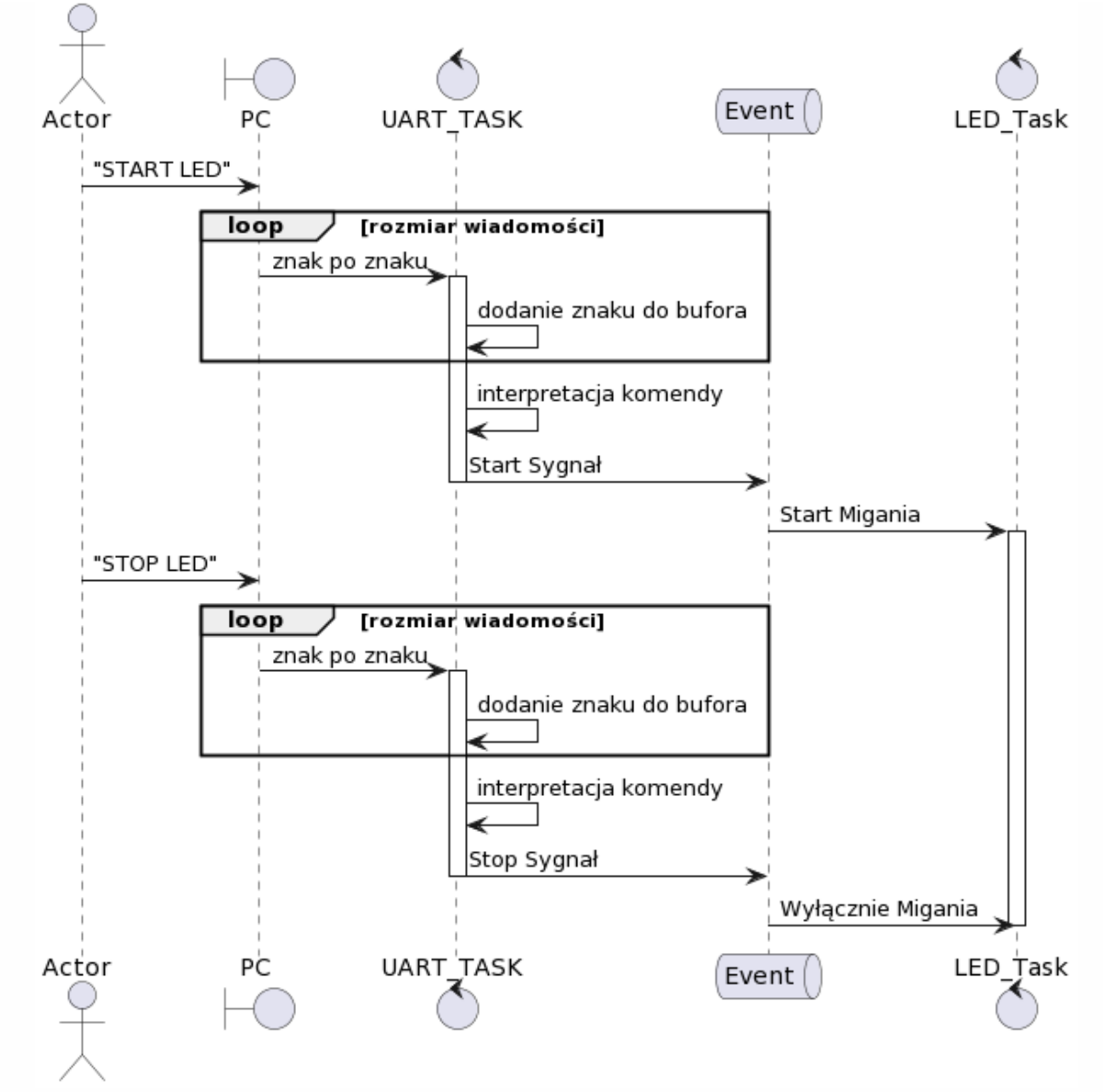
Projekt część (komunikacja plus interpreter): Karta Akwizycji w oparciu o system operacyjny

Celem jest przygotowanie platformy z mikrokontrolerem STM32 do dalszej pracy nad projektem końcowym - kartą akwizycji danych. Komputer, który będzie się komunikował z płytką mikrokontrolera po porcie szeregowym, będzie wysyłał różne rozkazy do zrealizowania. Kod mikrokontrolera będzie oparty na frameworku FreeRTOS. Dlatego w pierwszej kolejności stworzymy task obsługujący komunikację z komputerem. Task ten będzie miał najwyższy priorytet, aby system zachował responsywność. Przyjmujemy, że znakiem końca linii oznaczającym koniec komendy jest **„\n”**. Ten znak będzie sygnałem dla mikrokontrolera o interpretacji wysłanej komendy. **Bufor znaków,** jakie mikrokontroler może przyjąć, wynosi 20. Jeśli w buforze nie znajdzie się **znak końca linii**, mikrokontroler odpowie „Syntax Error” i zwróci bufor. Jeśli komenda jest poprawna, to wyśle sygnał (event, semafor lub kolejka) do odpowiedniego zadania.

Rysunek przedstawiający obsługę błędów:



Dla przykładu rozpoczniemy od prostej implementacji migania diodą. Program będzie składał się z dwóch tasków: Task UART_Commands (propozycja nazwy) oraz Task LED_Blinking(propozycja nazwy). Jeżeli Task UART_Commands otrzyma komendę **„START LED\n”**, to wyzwoli to Task LED_Blinking (częstotliwość dowolna). Jeśli otrzyma on komendę **„STOP LED\n”**, to zatrzyma Task odpowiedzialny za miganie diodą.



Uwaga: projekt będzie rozwijany

- W kolejnych etapach zostaną dodane nowe **komendy tekstowe** oraz **nowe taski**, realizujące dodatkowe funkcje.
- Architektura powinna być napisana tak, aby łatwo było ją **rozszerzać** o kolejne funkcje i zadania.
- Warto zaprojektować kod w sposób **modularny** i przejrzysty – z myślą o dalszym rozwoju.

Wymagania do realizacji projektu (część – komunikacja):

- ☐ **Użyj mikrokontrolera STM32** lub podobnego oraz frameworka **FreeRTOS.**
- ☐ **Stwórz dwa taski**:
  - UART_Commands – odbierający i interpretujący komendy tekstowe z komputera przez port szeregowy (UART).
  - LED_Blinking – odpowiedzialny za miganie diodą po otrzymaniu odpowiedniej komendy.
- ☐ **Zaimplementuj bufor wejściowy** na maksymalnie **20 znaków.**
- ☐ **Zakończenie komendy** powinno być oznaczane znakiem „\n”.
- ☐ Jeśli bufor nie zawiera „\n” przed jego przepełnieniem – system powinien odpowiedzieć komunikatem **"Syntax Error"** oraz odesłać zawartość bufora.
- ☐ W przypadku poprawnej komendy:
  - START LED\n → uruchom task LED_Blinking.
  - STOP LED\n → zatrzymaj task LED_Blinking.
- ☐ **Dobierz mechanizm synchronizacji** do komunikacji między taskami (np. kolejka, semafor, event group) i pamiętaj o uzasadnieniu **swojego wyboru.**
- ☐ Task odbierający komendy (UART_Commands) powinien mieć **najwyższy priorytet**, aby zapewnić responsywność systemu.
- ☐ System powinien zwracać stosowne komunikaty (np. potwierdzenia, błędy) przez UART do komputera.
- ☐ **Prędkość odbierania danych z UART może być bardzo wysoka**, znacznie większa niż w przypadku ręcznego wpisywania komend z klawiatury. **System powinien być w stanie obsłużyć szybki napływ danych** – np. przy pomocy przerwań, DMA lub buforu cyklicznego (ring buffer).
- ☐ **Skonfiguruj interfejs UART** z następującymi parametrami:
  - Baudrate: **115200 bps**
  - Długość słowa: **8 bitów**
  - Bit stopu: **1**
  - Parzystość: **brak (None)**

Wskazówki:

Odbieraj dane UART w trybie przerwaniowym lub DMA: Zamiast aktywnie czekać na znaki (polling), np. skonfiguruj UART do odbioru bajtów za pomocą HAL_UART_Receive_IT.

Oddziel logikę UART od logiki akcji: Zamiast wykonywać wszystko w przerwaniu UART, **przekaż gotową komendę do taska.**

Nie blokuj tasków zbyt długo: Unikaj długich operacji w taskach o wysokim priorytecie. Task komunikacyjny powinien być szybki i responsywny.

Zaprojektuj kod modularnie: Zamiast pisać wszystko w jednym pliku, warto rozdzielić obsługę UART, parser komend i logikę LED.

Do debugowania używaj SWD

Do porównywania ciągów znaków możesz użyć **if(strcmp(cmd_buffer, "START_LED") == 0).** W wersji bardziej profesjonalnej warto zamiast „ifologi strcmp” używać strncmp + parsować komendy do struktury/enuma i wtedy switch case.